AMRITA VISHWA VIDYAPEETHAM

DEEMED TO BE UNIVERSITY

DATASTRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

SUBMITTED BY	
NAME	DIVYA SAHITHI MASINENI
ROLL NO	AV.SC.U4CSE25129
YEAR/SEM/SECTION	2ndYEAR/ SEM-3/CSE-B
CREDITS	05

SUBMITTED TO	DR. RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFFESSOR(SL Grade)
DEPARTMENT	SCHOOL OF COMPUTING

INDEX

S.NO	TITLE	PAGE NO	DATE
1	**WEEK 1:** **Recursion** A) Recursive program for countdown from n to 1, followed by 'launch'. B) Recursive program to calculate P^n (compound interest growth factor P, years n). C) Recursive program to print first n terms of Fibonacci series. D) Recursive program to search for an employee ID in a list. E) Recursive program to calculate factorial (n!) for parcel arrangement.	5-11	14-07-26
2.	**WEEK 2:** **Searching** A) Python program to perform Linear Search on given input. B) Python program to perform Binary Search on given input (verify with sorted and unsorted lists).	12-16	21-07-26
3	**WEEK 3:** **Elementary Sorting** A) Python program to implement Bubble Sort (user input). B) Python program to implement Selection Sort (user input). C) Python program to implement Insertion Sort (user input).	17-21	01-08-26

INDEX

S.NO	TITLE	PAGE NO	DATE
4	**WEEK 4:** **Advanced Sorting** A) Python program to implement Merge Sort (user input). B) Python program to implement Quick Sort (user input).	22-25	11-08-26
5.	**WEEK 5:** **Linked Lists** Implement operations for: A) Singly Linked List (SLL) B) Doubly Linked List (DLL) C) Circular Linked List (CLL) Operations: 1. Create LL 2. Insert at Beginning 3. Insert at End 4. Insert at Index 5. Delete by Value 6. Delete Prior Node 7. Delete Last Node 8. Count Nodes 9. Display/Traverse 10. Exit	26-59	18-08-26
6	**WEEK 6:** **Stacks** A) Implement Stack operations (Push, Pop, Peek, Display) using Array. B) Implement Stack operations (Push, Pop, Peek, Display) using Linked List.	60-67	25-08-26

INDEX

S.NO	TITLE	PAGE NO	DATE
7.	**WEEK 7:** **Queues** A) Implement Queue using Arrays. B) Implement Queue using Linked List. C) Implement Circular Queue using Arrays. D) Implement Circular Queue using Linked List.	68-85	22-09-26
8.	**WEEK 8:** **Binary Trees** A) Implement Binary Tree with operations using Array. B) Implement Binary Tree with operations using Linked List.	86-89	29-09-26
9.	**WEEK 9:** **Binary Search Trees** Python program to implement Binary Search Tree (BST) operations.	90-94	06-10-26
10.	**WEEK 10:** **AVL TREE** Write a Python program to implement an AVL Tree with all its operations.	95-104	13-10-26
11.	**WEEK 11:** **Hashing** Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing).	105-108	27-10-26

LAB-1

1.AIM: A rocket launch system displays a countdown before launch. Instead of using a loop implement the countdown using recursion. Write a recursion python program to display a countdown from n to 1 followed by “launch”.

PROGRAM:

```
def launch(n):
 for i in range(n,-1,-1):
  if i==0:
   print("launch")
 else:
   print(i)
n=int (input ("Enter  countdown number:"))
launch(n)
```

OUTPUT:

```
============ RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab1a.py ==========
Enter countdown number: 7
7
6
5
4
3
2
1
Launch
```

EXPLANATION:

This program displays a countdown from n to 1 and finally prints 'launch'. Although the aim mentions recursion, the given code uses a loop

ERROR TABLE:

S.no.	Errors	Rectification
1.	Indentation Error	Use 3 **spaces** for each level of indentation.
2.	Type Error	Convert the input to an integer using n = int (input ("Enter no. of seconds to launch: ")).

2.AIM: The banks calculate the growth of an investment using compound interest; the formula requires calculating powers. Which can be implemented recursively. Write a recursive program to calculate p^n(power), where p=principal growth factor, n= number of years.

PROGRAM:

```
def power (p, n):
 if n == 0:
  return 1
 else:
   return p * power (p, n - 1)
p = float (input ("Enter principal/base value: "))
n = int (input ("Enter number of years/powers: "))
result = power (p, n)
print("Power =", result)
```

OUTPUT:

```
============ RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab1b.py ==========
Enter principal/base value: 456
Enter number of years/power: 23
Result = 1432819163011061360894656451138504049716965586066841347398041 6
```

EXPLANATION:

- The function of power (p, n) calculates p^n using recursion by multiplying p with the result of power (p, n-1).
- The recursion stops when n becomes 0, returning 1, which is the base case.
- The final result is displayed after all recursive calls are completed.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Type Error	Convert the input to float or int using float(input()) or int(input()).
2.	Incorrect output if the recursive call is written as power(p, n + 1) instead of power(p, n - 1).	Change the recursive call to power(p, n - 1) so that n decreases toward the base case.

3.AIM: An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

PROGRAM:

```
def search (list, key, index):
 if index >= len(list):
  return False
 if list[index] == key:
  return True
 return search (lst, key, index + 1)
employees = [101, 102, 103, 104, 105]
key = int (input ("Enter Employee ID to search: "))
```

```
if search (employees, key, 0):
print (f"Employee ID found at position {key}")
else:
print ("Employee ID Not Found")
```

OUTPUT:

```
============ RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab1c.py =========
Enter employee ID to search: 105
Employee ID found at position 5


============ RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab1c.py =======
Enter employee ID to search: 321
Employee ID not found
```

EXPLANATION:

- The recursive function checks each employee ID in the list one by one, starting from the first element.
- If the required ID is found, it returns True; otherwise, it continues searching recursively until the end of the list.
- If the entire list is searched without finding the ID, the function returns False.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Name Error if the list name is misspelled (e.g., employee instead of employees).	Use the correct variable name consistently throughout the program.
2.	Syntax Error due to a missing colon (:) after an if statement or function definition.	Add the missing colon (:) after if, else, or def.

4.AIM: In a logistics company the number of possible ways to arrange n different parcels are calculated using the factorial formula(n!). develop a recursive program to calculate the factorial of a given number.

PROGRAM:

```
def factorial(n):
  if n == 0 or n == 1:
    return 1
  else:
    return n * factorial (n - 1)
num = int (input ("Enter a number: "))
if num < 0:
  print ("Factorial does not exist for negative numbers.")
else:
  print ("Factorial =", factorial(num))
```

OUTPUT:

```
============ RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab1d.py =======
Enter a number: 14
Factorial = 87178291200
```

EXPLANATION:

- The function factorial(n) computes the factorial by multiplying the current number with the factorial of the previous number.
- The recursion ends when n becomes 0 or 1, where the factorial value is 1.
- The program also checks for negative inputs since factorial is defined only for non-negative integers.

ERROR TABLE:

S.no.	Errors	Rectification
1.	IndentationError if the statements inside the function are not properly indented.	Maintain proper indentation (usually 4 spaces) inside the function.
2.	TypeError if a decimal number (e.g., 5.5) is entered for factorial calculation.	Enter only a whole number or convert the input using int().

5.AIM: Write a recursive python program to print the first n terms of the Fibonacci series.

PROGRAM:

```
def fibonacci(n):
  if n == 0:
    return 0
  elif n == 1:
    return 1
  else:
    return fibonacci(n - 1) + fibonacci(n - 2)
n = int (input ("Enter the number of terms: "))
print ("Fibonacci Series:")
for i in range(n):
  print (fibonacci(i), end=" ")
```

OUTPUT:

```
========== RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lable.py ==========
Enter number of terms: 32
Fibonacci series:
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4181 6765 10946 17711
 28657 46368 75025 121393 196418 317811 514229 832040 1346269
```

EXPLANATION:

- The function Fibonacci(n) returns the nth Fibonacci number by recursively adding the previous two Fibonacci numbers.
- The base cases return 0 for the first term and 1 for the second term.
- A for loop calls the recursive function repeatedly to print the first n terms of the Fibonacci series.

ERROR TABLE:

S.no.	Errors	Rectification
1.	Recursion Error if the base cases (n == 0 and n == 1) are not included.	Add both base conditions before the recursive call.
2.	Value Error if the user enters text instead of an integer.	Enter a valid integer for the number of terms.

LAB-2

1.AIM: Write a program to perform linear search on a given input.

PROGRAM:

```
def linear_search(arr,key):
    for i in range(len(arr)):
        if arr[i]==key:
            return i
        else:
            return -1
 n = int(input("Enter no. of elements:"))
arr=[]
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
key=int(input("Enter element to search:"))
result=linear_search(arr,key)
if result!=-1:
    print("Element found at index",result)
else:
    print("Element not found.")
```

OUTPUT:

```
========== RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab2a.py ==========
Enter elements: 43 54 65 34 65 321 76 3
Enter element to search: 321
Element found at position 6
```

EXPLANATION:

The linear_search() function checks each element of the list one by one and compares it with the given key.

- If the key is found, its index is returned; otherwise, the function returns -1.
- The program takes the array and search element as input and displays whether the element is found.

ERROR TABLE:

S.no.	Errors	Rectification
1.	ValueError occurs if a non-integer value is entered when int(input()) is used.	Enter only integer values for the number of elements and array elements.
2.	TypeError may occur if len(arr) is used on a variable that is not a list.	Make sure arr is initialized as a list using arr = [] and elements are appended to it.

2.AIM: Write a Python program to perform binary search on the given input. Verify by taking sorted array and unsorted array as input.

PROGRAM:

```
def binary_search(arr,key):
    low=0
    high=len(arr)-1
```

```
    while low<=high:
    mid=(low+high)//2
    if arr[mid]==key:
         return mid
    elif arr[mid]<key:
        low=mid+1
    else:
        high=mid-1
        return -1
n = int(input("Enter no. of elements:"))
arr=[]
print("Enter the elements:")
for i in range(n):
   arr.append(int(input()))
key=int(input("Enter element to search:"))
result=binary_search(arr,key)
if result!=-1:
   print("Element found at index",result)
else:
  print("Element not found.")
```

OUTPUT:

```
========== RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab2a.py ==========
Enter elements: 43 54 65 34 65 321 76 3
Enter element to search: 321
Element found at position 6
```

PROGRAM:

```
def binary_search(arr,key):
    low=0
    high=len(arr)-1
    while low<=high:
    mid=(low+high)//2
     if arr[mid]==key:
           return mid
     elif arr[mid]<key:
         low=mid+1
    else:
       high=mid-1
       return -1
n = int(input("Enter no. of elements:"))
arr=[]
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
 if arr==sorted(arr):
    print("\n The input list is already sorted.")
else:
```

```
print("\n The input list is not sorted.")
print("Sorting the list...")
arr.sort()
print("Sorted list.",arr)
key=int(input("Enter element to search:"))
result=binary_search(arr,key)
if result!=-1:
    print("Element found at index",result)
else:
    print("Element not found.")
```

OUTPUT:

```
========== RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab2b2.py =========
Enter elements: 542 43 53 756 423 67 342
Sorted array: [43, 53, 67, 342, 423, 542, 756]
Enter element to search: 43
Element found at position 1
```

EXPLANATION:

- The binary_search() function repeatedly divides the sorted list into two halves and compares the middle element with the search key.
- If the middle element is smaller than the key, the search continues in the right half; otherwise, it searches the left half.
- The second program verifies whether the input is sorted and sorts it using arr.sort() if necessary before performing binary search.

ERROR TABLE:

S.no.	Errors	Rectification
1.	IndexError can occur if mid is calculated or the low/high boundaries are modified incorrectly.	Initialize low = 0, high = len(arr) - 1 and update them as low = mid +1 or high = mid - 1.

Lab-3

1.AIM: write programs for the Bubble sort

PROGRAM:

```
#bubble sort
def bubble_sort(arr):
    n = len(arr)
    for i in range (n-1):
        for j in range(n-i-1):
            if arr[j]>arr[j+1]:
                arr[j],arr[j+1] = arr[j+1],arr[j]
    return arr
#taking input
n = int(input("Enter number of elements: "))
arr=[]
print("Enter elements: ")
for i in range(n):
    arr.append(int(input()))
bubble_sort(arr)
print("sorted array: ")
for element in arr:
    print(element, end=" ")
```

Output:

```
========== RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab3a.py ==========
Enter number of elements: 5
Enter elements:
45
3
77
56
54
sorted array:
3 45 54 56 77
```

Explanation:

Bubble sort compares two adjacent elements. If the left element is greater than the right element, they are swapped. After each pass, the largest remaining element moves to the end.

Error table:

S.no	Error	Correction
1.	Used wrong loop range	Use for j in range(n-i-1):
2.	Swap condition is wrong	Use if arr[j] > arr[j+1]:

2.AIM: write programs for the Insertion sort

PROGRAM:

```
# Insertion Sort

def insertion_sort(arr):

    n = len(arr)

    for i in range(1, n):

        key = arr[i]

        j = i - 1

        while j >= 0 and arr[j] > key:

            arr[j + 1] = arr[j]

            j = j - 1
```

```
        arr[j + 1] = key
    return arr
# Taking input
n = int(input("Enter number of elements: "))
arr = []
print("Enter elements: ")
for i in range(n):
    arr.append(int(input()))
insertion_sort(arr)
print("Sorted array: ")
for element in arr:
    print(element, end=" ")   same for this
```

Output:

```
============ RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab3b.py =====
Enter number of elements: 7
Enter elements:
65
234
76
5
56
45
76
Sorted array:
5 45 56 65 76 76 234
```

Explanation:

Insertion sort builds the sorted array one element at a time.

It takes one element called the **key**, compares it with the elements before it, shifts larger elements to the right, and inserts the key in the correct position.

Error table:

S.no	Error	Correction
1.	key is not stored properly	Use key = arr[i]
2.	arr[j+1] = key placed inside while loop	Place it after the while loop

3.AIM: write programs for the Bubble sort

PROGRAM:

```
# Selection Sort
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]
    return arr
# Taking input
n = int(input("Enter number of elements: "))
arr = []
print("Enter elements: ")
for i in range(n):
```

```
    arr.append(int(input()))
selection_sort(arr)
print("Sorted array: ")
for element in arr:
    print(element, end=" ")
```

Output

Explanation:

Selection sort repeatedly finds the **smallest element** from the unsorted part of the array and places it in its correct position.

Error table:

S.no	Error	Correction
1.	min_index is not initialized	Use min_index = i
2.	Swap is placed inside inner loop	Place swap after the inner loop

LAB-4

1.Aim: Write a python program to implement merge sort where the input elements are taken from the user

PROGRAM:

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]
        merge_sort(left)
        merge_sort(right)
        i = j = k = 0
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1
```

```
    while j < len(right):
        arr[k] = right[j]
        j += 1
        k += 1
# Input from user
arr = list(map(int, input("Enter elements separated by spaces: ").split()))
merge_sort(arr)
print("Sorted array:", arr)
```

Output:

```
>
============ RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab4a.py ===========
Enter elements separated by spaces: 43 45 35 9834 34 7  983 24 76
Sorted array: [7, 24, 34, 35, 43, 45, 76, 983, 9834]
>
============ RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab4a.py ===========
Enter elements separated by spaces: 46578908756432456
Sorted array: [46578908756432456]
>
```

Explanation:

Merge Sort is a **divide-and-conquer sorting algorithm**.It works in three main steps:

1. Divide – Divide the array into two halves.
2. Conquer – Recursively sort both halves.
3. Merge – Merge the two sorted halves into one sorted array.

Error table:

S.no	Error	Correction
1.	Wrong base condition	Use if len(arr) > 1 for splitting
2.	Wrong midpoint	Use mid = len(arr)//2
3.	Forgetting remaining elements	Append remaining elements

2.Aim: Write a python program to implement quick sort where the input elements are taken from the user

Program:

```
def quick_sort(arr, low, high):
    if low < high:
        pivot = arr[high]
        i = low - 1
        for j in range(low, high):
            if arr[j] < pivot:
                i += 1
                arr[i], arr[j] = arr[j], arr[i]
        arr[i + 1], arr[high] = arr[high], arr[i + 1]
        p = i + 1
        quick_sort(arr, low, p - 1)
        quick_sort(arr, p + 1, high)
# Input from user
arr = list(map(int, input("Enter elements separated by spaces: ").split()))
quick_sort(arr, 0, len(arr) - 1)
print("Sorted array:", arr)
```

Output:

```
============ RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab4b.py ==========
Enter elements separated by spaces: 43 234 45 231 78  23 65
Sorted array: [23, 43, 45, 65, 78, 231, 234]

============ RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab4b.py ==========
Enter elements separated by spaces: 354679874
Sorted array: [354679874]
```

Explanation:

Quick Sort is also a **divide-and-conquer sorting algorithm**.

Its main idea is to select an element called a **pivot** and arrange the array so that:

Elements smaller than pivot | Pivot | Elements greater than pivot

Then Quick Sort is recursively applied to the left and right portions.

Error table:

S.No.	Error	Correction
1	Incorrect Python syntax, missing ： or brackets	Check syntax, brackets and colons
2	Incorrect low/high boundaries are passed	Use low, pivot-1 and pivot+1, high
3	Function keeps calling itself	Add if low < high:

LAB-5

1.Aim: Write a Python program to implement a singly linked list with the below operations:

1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at a specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class SinglyLinkedList:
    def __init__(self):
        self.head = None
    # 1. Create linked list
    def create(self):
        n = int(input("Enter number of nodes: "))
        for i in range(n):
            data = int(input(f"Enter data for node {i + 1}: "))
  new_node = Node(data)
            if self.head is None:
                self.head = new_node
```

```
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new_node
    print("Linked list created successfully.")
# 2. Insert at beginning
def insert_beginning(self):
    data = int(input("Enter data: "))
    new_node = Node(data)
    new_node.next = self.head
    self.head = new_node
    print("Node inserted at beginning.")
# 3. Insert at end
def insert_end(self):
    data = int(input("Enter data: "))
    new_node = Node(data)
    if self.head is None:
        self.head = new_node
    else:
        temp = self.head
```

```
while temp.next:
            temp = temp.next
        temp.next = new_node
    print("Node inserted at end.")
# 4. Insert at specific index
def insert_index(self):
    index = int(input("Enter index: "))
    data = int(input("Enter data: "))
    new_node = Node(data)
    if index == 0:
        new_node.next = self.head
        self.head = new_node
        print("Node inserted.")
        return
    temp = self.head
    for i in range(index - 1):
        if temp is None:
            print("Invalid index.")
            return
        temp = temp.next
    if temp is None:
        print("Invalid index.")
        return
```

```
 new_node.next = temp.next
    temp.next = new_node
    print("Node inserted.")
# 5. Delete by value
def delete_value(self):
    value = int(input("Enter value to delete: "))
    if self.head is None:
        print("List is empty.")
        return
    if self.head.data == value:
        self.head = self.head.next
        print("Node deleted.")
        return
    temp = self.head
    while temp.next and temp.next.data != value:
        temp = temp.next
    if temp.next is None:
        print("Value not found.")
    else:
        temp.next = temp.next.next
        print("Node deleted.")
# 6. Delete first node
def delete_first(self):
```

```
    if self.head is None:
        print("List is empty.")
else:
        self.head = self.head.next
        print("First node deleted.")
 # 7. Delete last node
 def delete_last(self):
    if self.head is None:
        print("List is empty.")
        return
    if self.head.next is None:
        self.head = None
        print("Last node deleted.")
        return
    temp = self.head
    while temp.next.next:
        temp = temp.next
    temp.next = None
    print("Last node deleted.")
 # 8. Count nodes
 def count(self):
    count = 0
    temp = self.head
    while temp:
```

```
    count += 1
            temp = temp.next
        print("Number of nodes:", count)
    # 9. Display / Traverse
    def display(self):
        if self.head is None:
            print("List is empty.")
            return
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")
# Main program
sll = SinglyLinkedList()
while True:
    print("\n--- SINGLY LINKED LIST ---")
    print("1. Create linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete first node")
    print("7. Delete last node")
```

```
print("8. Count number of nodes")
print("9. Display / Traverse")
print("10. Exit")
choice = int(input("Enter your choice: "))
if choice == 1:
    sll.create()
elif choice == 2:
    sll.insert_beginning()
elif choice == 3:
    sll.insert_end()
elif choice == 4:
    sll.insert_index()
elif choice == 5:
    sll.delete_value()
elif choice == 6:
    sll.delete_first()
elif choice == 7:
    sll.delete_last()
elif choice == 8:
    sll.count()
elif choice == 9:
    sll.display()
elif choice == 10:
    print("Program exited.")
```

```
    break

else:

    print("Invalid choice.")
```

OUTPUT:

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (AMD
64)] on win32
Enter "help" below or click "Help" above for more information.
>

============ RESTART: C:/Users/Divya/OneDrive/Documents/DSA/lab5a.py ===========

--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter number of nodes: 5
Enter data for node 1: 54
Enter data for node 2: 54
Enter data for node 3: 32
Enter data for node 4: 2
Enter data for node 5: 6
Linked list created successfully.

--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 2
Enter data: 43
Node inserted at beginning.
```

```
--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 3
Enter data: 32
Node inserted at end.

--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 4
Enter index: 2
Enter data: 323
Node inserted.

--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
```

```
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 5
Enter value to delete: 776
Value not found.

--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 5
Enter value to delete: 323
Node deleted.

--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 6
First node deleted.

--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
```

```
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 7
Last node deleted.

--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 8
Number of nodes: 5

--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 9
54 -> 54 -> 32 -> 2 -> 6 -> None

--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
```

```
--- SINGLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 10
Program exited.
```

Explanation:

Singly Linked List – Explanation

A **singly linked list** is a linear data structure in which elements are stored in **nodes**. Each node contains two parts:

1. **Data** – stores the value.
2. **Next** – stores the address/reference of the next node.

- self.data stores the node value.
- self.next stores the reference to the next node.
- Initially, next = None because the node is not connected to another node.

Error table:

S.No	Mistake	Correction
1	`Node` class is not defined before using it	Define the `Node` class first
2	`temp` is not updated inside the loop	Add `temp = temp.next`
3	`head` is not updated	Use `self.head = new_node`
4	User enters an index outside the list range	Validate the index before insertion
5	The given value does not exist	Search the entire list and display "Value not found"
6	`head` is `None` and operation is performed directly	Check `if self.head is None`

2.Aim: Write a Python program to implement a Double linked list with the above operations

Program:

```
class Node:
 def __init__(self, data):
    self.data = data
    self.prev = None
    self.next = None
```

```
class DoublyLinkedList:
    def __init__(self):
        self.head = None
    # 1. Create linked list
    def create(self):
        n = int(input("Enter number of nodes: "))
        for i in range(n):
            data = int(input(f"Enter data for node {i + 1}: "))
            new_node = Node(data)
            if self.head is None:
                self.head = new_node
            else:
                temp = self.head
                while temp.next:
                    temp = temp.next
                temp.next = new_node
                new_node.prev = temp
        print("Doubly linked list created successfully.")
    # 2. Insert at beginning
    def insert_beginning(self):
        data = int(input("Enter data: "))
        new_node = Node(data)
```

```
    new_node = Node(data)
    if self.head is not None:
        new_node.next = self.head
        self.head.prev = new_node
    self.head = new_node
    print("Node inserted at beginning.")
# 3. Insert at end
def insert_end(self):
    data = int(input("Enter data: "))
    new_node = Node(data)
    if self.head is None:
        self.head = new_node
    else:
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new_node
        new_node.prev = temp
    print("Node inserted at end.")
# 4. Insert at specific index
def insert_index(self):
    index = int(input("Enter index: "))
    data = int(input("Enter data: "))
```

```
    new_node = Node(data)
    if index == 0:
        if self.head:
            new_node.next = self.head
            self.head.prev = new_node
        self.head = new_node
        print("Node inserted.")
        return
    temp = self.head
    for i in range(index - 1):
        if temp is None:
            print("Invalid index.")
            return
        temp = temp.next
    if temp is None:
        print("Invalid index.")
        return
    new_node.next = temp.next
    new_node.prev = temp
    if temp.next:
        temp.next.prev = new_node
    temp.next = new_node
    print("Node inserted.")
```

```
# 5. Delete by value
  def delete_value(self):
      value = int(input("Enter value to delete: "))
      temp = self.head
      while temp and temp.data != value:
          temp = temp.next
      if temp is None:
          print("Value not found.")
          return
      if temp.prev:
          temp.prev.next = temp.next
      else:
          self.head = temp.next
      if temp.next:
          temp.next.prev = temp.prev
      print("Node deleted.")
  # 6. Delete first node
  def delete_first(self):
      if self.head is None:
          print("List is empty.")
          return
      self.head = self.head.next
      if self.head:
          self.head.prev = None
```

```
print("First node deleted.")

# 7. Delete last node

def delete_last(self):

    if self.head is None:

        print("List is empty.")

        return

    temp = self.head

    while temp.next:

        temp = temp.next

    if temp.prev:

        temp.prev.next = None

    else:

        self.head = None

    print("Last node deleted.")

# 8. Count nodes

def count(self):

    count = 0

    temp = self.head

    while temp:

        count += 1

        temp = temp.next

    print("Number of nodes:", count)

# 9. Display / Traverse

def display(self):
```

```
        if self.head is None:
            print("List is empty.")
            return
        temp = self.head
        while temp:
            print(temp.data, end=" <-> ")
            temp = temp.next
        print("None")
# Main program
dll = DoublyLinkedList()
while True:
    print("\n--- DOUBLY LINKED LIST ---")
    print("1. Create linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete first node")
    print("7. Delete last node")
    print("8. Count number of nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))
```

```
if choice == 1:
    dll.create()
elif choice == 2:
    dll.insert_beginning()
elif choice == 3:
    dll.insert_end()
elif choice == 4:
    dll.insert_index()
elif choice == 5:
    dll.delete_value()
elif choice == 6:
    dll.delete_first()
elif choice == 7:
    dll.delete_last()
elif choice == 8:
    dll.count()
elif choice == 9:
    dll.display()
elif choice == 10:
    print("Program exited.")
    break
else:
    print("Invalid choice.")
```

Output:

```
============ RESTART: C:/Users/Divya/OneDrive/Documents/DSA/lab5b.py ===========

--- DOUBLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter number of nodes: 6
Enter data for node 1: 43
Enter data for node 2: 76
Enter data for node 3: 2
Enter data for node 4: 87
Enter data for node 5: 21
Enter data for node 6: 8
Doubly linked list created successfully.

--- DOUBLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 2
Enter data: 434

Node inserted at beginning.

--- DOUBLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 3
Enter data: 323
Node inserted at end.

--- DOUBLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 34
Invalid choice.

--- DOUBLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
```

```
9. Display / Traverse
10. Exit
Enter your choice: 4
Enter index: 4
Enter data: 33232
Node inserted.

--- DOUBLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 5
Enter value to delete: 34
Value not found.

--- DOUBLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 5
Enter value to delete: 87
Node deleted.

--- DOUBLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
```

```
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 6
First node deleted.

--- DOUBLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 7
Last node deleted.

--- DOUBLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 8
Number of nodes: 6

--- DOUBLY LINKED LIST ---
```

```
--- DOUBLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 9
43 <-> 76 <-> 2 <-> 33232 <-> 21 <-> 8 <-> None

--- DOUBLY LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 10
Program exited.
```

Explanation:

A **Doubly Linked List** is a linear data structure in which each node contains **three parts**:

1. prev – reference to the previous node
2. data – stores the value
3. next – reference to the next node

head points to the first node.

The first node's prev is None.

The last node's next is None.

Each node can be traversed **forward and backward**.

S.No	Error	Correction
1	Trying to access prev of a nonexistent node	Use an appropriate if condition
2	prev links are not updated	Correctly update prev references
3	head becomes None	Check whether the list becomes empty
4	Second-last node's next is not changed	Set temp.prev.next = None
5	Index is outside the list	Validate the index

2.Aim: Write a Python program to implement a Circular linked list with the above operations

Program:

```
class Node:
  def __init__(self, data):
     self.data = data
     self.next = None
class CircularLinkedList:
    def __init__(self):
         self.head = None
```

```
# 1. Create linked list
def create(self):
    n = int(input("Enter number of nodes: "))
    for i in range(n):
        data = int(input(f"Enter data for node {i + 1}: "))
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            new_node.next = self.head
        else:
            temp = self.head
            while temp.next != self.head:
                temp = temp.next
            temp.next = new_node
            new_node.next = self.head
    print("Circular linked list created successfully.")
# 2. Insert at beginning
def insert_beginning(self):
    data = int(input("Enter data: "))
    new_node = Node(data)
    if self.head is None:
        self.head = new_node
        new_node.next = self.head
```

```
    else:
        temp = self.head
        while temp.next != self.head:
            temp = temp.next
        new_node.next = self.head
        temp.next = new_node
        self.head = new_node
    print("Node inserted at beginning.")
# 3. Insert at end
def insert_end(self):
    data = int(input("Enter data: "))
    new_node = Node(data)
    if self.head is None:
        self.head = new_node
        new_node.next = self.head
    else:
        temp = self.head
        while temp.next != self.head:
            temp = temp.next
        temp.next = new_node
        new_node.next = self.head

    print("Node inserted at end.")
```

```
# 4. Insert at specific index
def insert_index(self):
    index = int(input("Enter index: "))
    data = int(input("Enter data: "))
    new_node = Node(data)
    if index == 0:
        if self.head is None:
            self.head = new_node
            new_node.next = self.head
        else:
            temp = self.head
            while temp.next != self.head:
                temp = temp.next
            new_node.next = self.head
            temp.next = new_node
            self.head = new_node
        print("Node inserted.")
        return
    if self.head is None:
        print("Invalid index.")
        return
    temp = self.head
    for i in range(index - 1):
        temp = temp.next
```

```
# 5. Delete by value
def delete_value(self):
    value = int(input("Enter value to delete: "))
    if self.head is None:
        print("List is empty.")
        return
    current = self.head
    previous = None
    while True:
        if current.data == value:
            break
        previous = current
        current = current.next
        if current == self.head:
            print("Value not found.")
            return
    # Only one node
    if current == self.head and current.next == self.head:
        self.head = None
    # Delete head
    elif current == self.head:
        last = self.head
        while last.next != self.head:
            last = last.next
```

```
            self.head = self.head.next
            last.next = self.head
        # Delete other node
        else:
            previous.next = current.next
        print("Node deleted.")
    # 6. Delete first node
    def delete_first(self):
        if self.head is None:
            print("List is empty.")
            return
        # Only one node
        if self.head.next == self.head:
            self.head = None
        else:
            last = self.head
            while last.next != self.head:
                last = last.next
            self.head = self.head.next
            last.next = self.head
        print("First node deleted.")
    # 7. Delete last node
    def delete_last(self):
        if self.head is None:
```

```
        print("List is empty.")
        return
    # Only one node
    if self.head.next == self.head:
        self.head = None
        print("Last node deleted.")
        return
    temp = self.head
    while temp.next.next != self.head:
        temp = temp.next
    temp.next = self.head
    print("Last node deleted.")
# 8. Count nodes
def count(self):
    if self.head is None:
        print("Number of nodes: 0")
        return
    count = 0
    temp = self.head
    while True:
        count += 1
        temp = temp.next
        if temp == self.head:
            break
    print("Number of nodes:", count)
```

```
    # 9. Display / Traverse
    def display(self):
        if self.head is None:
            print("List is empty.")
            return
        temp = self.head
        while True:
            print(temp.data, end=" -> ")
            temp = temp.next
            if temp == self.head:
                break
        print("(back to head)")
# Main program
cll = CircularLinkedList()
while True:
    print("\n--- CIRCULAR LINKED LIST ---")
    print("1. Create linked list")
    print("2. Insert at beginning")
    print("3. Insert at end")
    print("4. Insert at specific index")
    print("5. Delete by value")
    print("6. Delete first node")
    print("7. Delete last node")
    print("8. Count number of nodes")
```

```
print("9. Display / Traverse")
print("10. Exit")
choice = int(input("Enter your choice: "))
if choice == 1:
    cll.create()
elif choice == 2:
    cll.insert_beginning()
elif choice == 3:
    cll.insert_end()
elif choice == 4:
    cll.insert_index()
elif choice == 5:
    cll.delete_value()
elif choice == 6:
    cll.delete_first()
elif choice == 7:
    cll.delete_last()
elif choice == 8:
    cll.count()
elif choice == 9:
    cll.display()
elif choice == 10:
    print("Program exited.")
    break
else:
    print("Invalid choice.")
```

Output:

```
>
========== RESTART: C:/Users/Divya/OneDrive/Documents/DSA/lab5c.py ==========

--- CIRCULAR LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter number of nodes: 7
Enter data for node 1: 657
Enter data for node 2: 543
Enter data for node 3: 45
Enter data for node 4: 3245
Enter data for node 5: 876
Enter data for node 6: 345
Enter data for node 7: 34456
Circular linked list created successfully.

--- CIRCULAR LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 2
Enter data: 32
Node inserted at beginning.

--- CIRCULAR LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 3
Enter data: 7654
Node inserted at end.

--- CIRCULAR LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 23
Invalid choice.

--- CIRCULAR LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
```

```
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 4
Enter index: 5
Enter data: 66665
Node inserted.

--- CIRCULAR LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 5
Enter value to delete: 754
Value not found.

--- CIRCULAR LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 45
Invalid choice.

--- CIRCULAR LINKED LIST ---
1. Create linked list
2. Insert at beginning
```

```
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 5
Enter value to delete: 45
Node deleted.

--- CIRCULAR LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 6
First node deleted.

--- CIRCULAR LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 7
Last node deleted.
```

```
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 8
Number of nodes: 7

--- CIRCULAR LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 9
657 -> 543 -> 3245 -> 66665 -> 876 -> 345 -> 34456 -> (back to head)

--- CIRCULAR LINKED LIST ---
1. Create linked list
2. Insert at beginning
3. Insert at end
4. Insert at specific index
5. Delete by value
6. Delete first node
7. Delete last node
8. Count number of nodes
9. Display / Traverse
10. Exit
Enter your choice: 10
Program exited.
```

Explanation:

A Circular Linked List is a linked list in which the **last node points back to the first node** instead of pointing to None.

New node points to the old head.

Last node points to the new head.

Error table:

S.No	Error	Correction
1	Infinite loop during traversal	Stop when `temp == head`
2	Circular connection is lost	Set last node's `next` to `head`
3	Insert at beginning breaks the circle	Update `last.next = new_node`
4	Count is incorrect	Stop when pointer reaches `head`
5	Invalid index	Check index before insertion

LAB-6

1.Aim: Write a Python program to implement stack operations such as **Push, Pop, Peek, Display** using an **array**

Program:

```
# Stack implementation using Array

stack = [543,45,324,435,34,45,32]

MAX_SIZE = 5

def push():

  if len(stack) == MAX_SIZE:

    print("Stack Overflow")

 else:

    element = int(input("Enter element to push: "))

     stack.append(element)

 print(element, "pushed into stack")

def pop():

 if len(stack) == 0:

    print("Stack Underflow")

 else:

    element = stack.pop()

 print(element, "popped from stack")

def peek():

if len(stack) == 0:

   print("Stack is empty")
```

```
  else:
     print("Top element is:", stack[-1])
def display():
 if len(stack) == 0:
    print("Stack is empty")
 else:
   print("Stack elements are:")
  for i in range(len(stack) - 1, -1, -1):
 print(stack[i])
Menu-driven program
while True:
       print("\n--- STACK USING ARRAY ---")
       print("1. Push")
       print("2. Pop")
       print("3. Peek")
       print("4. Display")
       print("5. Exit")
       choice = int(input("Enter your choice: "))
        if choice == 1:
            push()
        elif choice == 2:
          pop()
        elif choice == 3:
           peek()
```

```
    elif choice == 4:
        display()
    elif choice == 5:
        print("Program terminated")
        break
    else:
        print("Invalid choice")
```

Output:

```
--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 654
Invalid choice

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 39
39 pushed into stack

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
39 popped from stack

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
```

```
--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element is: 543

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements are:
543
45
324
435
34
45
32

--- STACK USING ARRAY ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated
```

Explanation:

A **stack** is a linear data structure that follows the **LIFO (Last In, First Out)** principle. The element inserted last is removed first.

The main stack operations are:

1. **Push** – Adds an element to the top of the stack.
2. **Pop** – Removes the top element from the stack.
3. **Peek** – Displays the top element without removing it.
4. **Display** – Displays all elements in the stack.

Error table:

S.No	Error	Correction
1	IndexError: pop from empty list	Check len(stack) == 0 before pop
2	Stack Overflow	Check len(stack) == MAX_SIZE
3	IndexError: list index out of range	Check whether stack is empty before Peek
4	ValueError	Enter a valid integer

2.Aim: Write a Python program to implement stack operations such as **Push, Pop, Peek, Display** using **a linked list**.

Program:

```
#Stack implementation using Linked List

class Node:

  def init(self, data):

    self.data = data

    self.next = None

class Stack:

 def init(self):

   self.top = None


  # Push operation
  def push(self):
     element = int(input("Enter element to push: "))

     new_node = Node(element)
     new_node.next = self.top
     self.top = new_node

     print(element, "pushed into stack")

  # Pop operation
  def pop(self):
     if self.top is None:
        print("Stack Underflow")
     else:
        element = self.top.data
        self.top = self.top.next
        print(element, "popped from stack")
```

```
# Peek operation
def peek(self):
    if self.top is None:
        print("Stack is empty")
    else:
        print("Top element is:", self.top.data)

# Display operation
def display(self):
    if self.top is None:
        print("Stack is empty")
    else:
        temp = self.top
        print("Stack elements are:")

        while temp is not None:
            print(temp.data)
            temp = temp.next

#Menu-driven program

stack = Stack()

while True:

        print("\n--- STACK USING LINKED LIST ---")

        print("1. Push")

        print("2. Pop")

        print("3. Peek")

        print("4. Display")

        print("5. Exit")
```

```
choice = int(input("Enter your choice: "))
if choice == 1:
    stack.push()
elif choice == 2:
    stack.pop()
elif choice == 3:
    stack.peek()
elif choice == 4:
    stack.display()
elif choice == 5:
    print("Program terminated")
    break
else:
    print("Invalid choice")
```

Output:

```
============ RESTART: C:/Users/Divya/OneDrive/Documents/DSA/lab6b.py =======

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 32
32 pushed into stack

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1987
Invalid choice

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 13
Invalid choice

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to push: 345
345 pushed into stack
```

```
Enter element to push: 345
345 pushed into stack

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
345 popped from stack

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element is: 32

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
Stack elements are:
32

--- STACK USING LINKED LIST ---
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated
```

Explanation:

A linked-list stack consists of **nodes**.

A temporary pointer travels through the linked list:

Error table:

S.No	Error	Correction
1	AttributeError	Check self.top is None before accessing data
2	Stack Underflow	Check whether top is None
4	NameError	Use the correct variable/reference

Lab-7

1.Aim: Write a Python program to implement a **queue** and its operations such as **Enqueue, Dequeue, Peek, Display** with a **menu-driven approach**. Implement using **array**

Program:

```
#Queue Implementation using Linked List

#Operations: Enqueue, Dequeue, Peek, Display

class Node:

  def init(self, data):

    self.data = data

    self.next = None

class Queue:

  def init(self):

    self.front = None

    self.rear = None


  # Enqueue operation
  def enqueue(self, data):
    new_node = Node(data)
    if self.rear is None:
      self.front = self.rear = new_node
    else:
      self.rear.next = new_node
      self.rear = new_node
    print(data, "inserted into queue.")
```

```
# Dequeue operation
def dequeue(self):
    if self.front is None:
        print("Queue Underflow! Queue is empty.")
        return
    value = self.front.data
    self.front = self.front.next
    if self.front is None:
        self.rear = None
    print(value, "deleted from queue.")
# Peek operation
def peek(self):
    if self.front is None:
        print("Queue is empty.")
    else:
        print("Front element:", self.front.data)

# Display operation
def display(self):
    if self.front is None:
        print("Queue is empty.")
        return
    current = self.front
    print("Queue elements:", end=" ")
    while current is not None:
        print(current.data, end=" ")
        current = current.next
    print()

queue = Queue()

while True:

print("\n----- QUEUE USING LINKED LIST -----")

print("1. Enqueue")

print("2. Dequeue")

print("3. Peek")

print("4. Display")

print("5. Exit")
```

```
choice = int(input("Enter your choice: "))
if choice == 1:
    value = int(input("Enter element to enqueue: "))
   queue.enqueue(value)
elif choice == 2:
    queue.dequeue()

elif choice == 3:
    queue.peek()
elif choice == 4:
    queue.display()

elif choice == 5:
    print("Program terminated.")
    break
else:
    print("Invalid choice! Please try again.")
```

Output:

```
============ RESTART: C:/Users/Divya/OneDrive/Documents/DSA/lab7a.py ===========

----- QUEUE USING ARRAY -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 765
765 inserted into queue.

----- QUEUE USING ARRAY -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 87867
87867 inserted into queue.

----- QUEUE USING ARRAY -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 187
Invalid choice! Please try again.

----- QUEUE USING ARRAY -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 543
543 inserted into queue.
```

```
5. Exit
Enter your choice: 2
765 deleted from queue.

----- QUEUE USING ARRAY -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 87867
Invalid choice! Please try again.

----- QUEUE USING ARRAY -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 87867

----- QUEUE USING ARRAY -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements: [87867, 543]

----- QUEUE USING ARRAY -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated.
```

Explanation:

A **queue** is a linear data structure that follows the **FIFO (First In, First Out)** principle.

In this program, Python's **list** is used as an array to store queue elements.

Error table:

S.No	Error	Rectification
1	Using queue.pop() instead of queue.pop(0) removes the last element instead of the front element.	Use queue.pop(0) for dequeue operation.
2	Not checking the queue size before insertion can cause the queue to exceed its maximum size.	Check if len(queue) == MAX_SIZE before enqueue.
3	Accessing queue[0] when the queue is empty causes an IndexError.	Check if len(queue) == 0 before performing Peek.

2.Aim:

Write a Python program to implement a **queue** and its operations such as **Enqueue, Dequeue, Peek, Display** with a **menu-driven approach**. Implement using **linked-list data structures**.

Program:

```
#Queue Implementation using Linked List

#Operations: Enqueue, Dequeue, Peek, Display

class Node:

  def init(self, data):

      self.data = data

      self.next = None

class Queue:

  def init(self):

       self.front = None

       self.rear = None
```

```
# Enqueue operation
def enqueue(self, data):
    new_node = Node(data)
    if self.rear is None:
        self.front = self.rear = new_node
    else:
        self.rear.next = new_node
        self.rear = new_node
    print(data, "inserted into queue.")
# Dequeue operation
def dequeue(self):
    if self.front is None:
        print("Queue Underflow! Queue is empty.")
        return

    value = self.front.data
    self.front = self.front.next

    if self.front is None:
        self.rear = None
    print(value, "deleted from queue.")
```

```
# Peek operation
def peek(self):
    if self.front is None:
        print("Queue is empty.")
    else:
        print("Front element:", self.front.data)
# Display operation
def display(self):
    if self.front is None:
        print("Queue is empty.")
        return
    current = self.front
    print("Queue elements:", end=" ")
    while current is not None:
        print(current.data, end=" ")
       current = current.next
    print()

queue = Queue()

while True:

        print("\n----- QUEUE USING LINKED LIST -----")

        print("1. Enqueue")

        print("2. Dequeue")

        print("3. Peek")

        print("4. Display")

        print("5. Exit")

        choice = int(input("Enter your choice: "))

        if choice == 1:
            value = int(input("Enter element to enqueue: "))
            queue.enqueue(value)
```

```
elif choice == 2:
    queue.dequeue()

elif choice == 3:
    queue.peek()

elif choice == 4:
    queue.display()

elif choice == 5:
    print("Program terminated.")
    break

else:
    print("Invalid choice! Please try again.")
```

Output:

```
============ RESTART: C:/Users/Divya/OneDrive/Documents/DSA/lab7b.py =======

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 65
65 inserted into queue.

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 787
787 inserted into queue.

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 987
987 inserted into queue.

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1857
```

```
Edit  Shell  Debug  Options  Window  Help
5. Exit
Enter your choice: 1857
Invalid choice! Please try again.

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
65 deleted from queue.

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 787

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Queue elements: 787 987

----- QUEUE USING LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated.
```

Explanation:

A **linked-list queue** consists of dynamically created nodes. Each node contains two parts:

Two pointers are maintained:

- **Front:** Points to the first node.
- **Rear:** Points to the last node.

Error table:

S.No	Error	Rectification
1	After deleting the last node, rear may still point to the deleted node.	If front becomes None, set self.rear = None.
2	Displaying the linked list without moving to the next node can create an infinite loop.	Use current = current.next inside the display loop.

3.Aim: Write a Python program to implement a **queue** and its operations such as **Enqueue, Dequeue, Peek, Display** with a **menu-driven approach**. Implement using **Circular Queue Arrays structures**.

Program:

```
#Circular Queue Implementation using Array

#Operations: Enqueue, Dequeue, Peek, Display

MAX_SIZE = 5

queue = [None] * MAX_SIZE

front = -1

rear = -1

while True:

      print("\n----- CIRCULAR QUEUE -----")

      print("1. Enqueue")

      print("2. Dequeue")

      print("3. Peek")

      print("4. Display")

      print("5. Exit")

  choice = int(input("Enter your choice: "))

  # Enqueue
  if choice == 1:

     # Check whether queue is full
     if (rear + 1) % MAX_SIZE == front:
        print("Queue Overflow! Circular Queue is full.")

     else:
        value = int(input("Enter element to enqueue: "))
```

```
# First element
        if front == -1:
            front = 0
            rear = 0
        else:
            rear = (rear + 1) % MAX_SIZE
        queue[rear] = value
        print(value, "inserted into circular queue.")

# Dequeue
elif choice == 2:

    if front == -1:
        print("Queue Underflow! Circular Queue is empty.")
    else:
        value = queue[front]
        print(value, "deleted from circular queue.")
        # Only one element was present
        if front == rear:
            front = -1
            rear = -1
        else:
            front = (front + 1) % MAX_SIZE
# Peek
elif choice == 3:

    if front == -1:
        print("Queue is empty.")
    else:
        print("Front element:", queue[front])
# Display
elif choice == 4:
    if front == -1:
        print("Queue is empty.")
    else:
        print("Circular Queue elements:", end=" ")
        i = front
```

```
while True:
        print(queue[i], end=" ")
        if i == rear:
            break

        i = (i + 1) % MAX_SIZE
    print()
# Exit
elif choice == 5:
    print("Program terminated.")
    break

else:
    print("Invalid choice! Please try again.")
```

Output:

```
========== RESTART: C:/Users/Divya/OneDrive/Documents/DSA/lab7c.py =====

----- CIRCULAR QUEUE -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 46
46 inserted into circular queue.

----- CIRCULAR QUEUE -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 87
87 inserted into circular queue.

----- CIRCULAR QUEUE -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 8756
8756 inserted into circular queue.

----- CIRCULAR QUEUE -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1786
Invalid choice! Please try again.
```

```
5. Exit
Enter your choice: 1786
Invalid choice! Please try again.

----- CIRCULAR QUEUE -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
46 deleted from circular queue.

----- CIRCULAR QUEUE -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 87

----- CIRCULAR QUEUE -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Circular Queue elements: 87 8756

----- CIRCULAR QUEUE -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated.
```

Explanation:

A **circular queue** is a queue in which the last position is connected back to the first position.

Unlike a normal linear queue, a circular queue can **reuse empty positions** created after dequeue operations.

Error table:

S.No	Error	Rectification
1	Using rear + 1 without % MAX_SIZE can make the rear go outside the array.	Use rear = (rear + 1) % MAX_SIZE.
2	Not resetting both pointers after deleting the last element can leave the queue in an incorrect state.	When front == rear, set both front = -1 and rear = -1.

4.Aim: Write a Python program to implement a **queue** and its operations such as **Enqueue, Dequeue, Peek, Display** with a **menu-driven approach**. Implement using **Circular Queue linked-list data structures**.

Program:

```
#Circular Queue using Singly Linked List

#Operations: Enqueue, Dequeue, Peek, Display

class Node:

  def init(self, data):

     self.data = data

      self.next = None

class CircularQueue:

  def init(self):

     self.rear = None


  # Enqueue operation
  def enqueue(self, data):
     new_node = Node(data)
     # Queue is empty
     if self.rear is None:
        self.rear = new_node
        self.rear.next = self.rear
     else:
        # New node is inserted after rear
        new_node.next = self.rear.next
        self.rear.next = new_node
        self.rear = new_node

     print(data, "inserted into circular queue.")
```

```
# Dequeue operation
def dequeue(self):
    if self.rear is None:
        print("Queue Underflow! Queue is empty.")
        return

    # Front node
    front = self.rear.next

    # Only one node
    if self.rear == front:
        self.rear = None

    else:
        self.rear.next = front.next

    print(front.data, "deleted from circular queue.")
# Peek operation
def peek(self):
    if self.rear is None:
        print("Queue is empty.")
    else:
        print("Front element:", self.rear.next.data)
# Display operation
def display(self):
    if self.rear is None:
        print("Queue is empty.")
        return

    front = self.rear.next
    current = front
    print("Circular Queue elements:", end=" ")
    while True:
        print(current.data, end=" ")
        current = current.next
        if current == front:
            break
    print()
```

```
#Create circular queue

queue = CircularQueue()

Menu-driven program

while True:

        print("\n----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----")

        print("1. Enqueue")

        print("2. Dequeue")

        print("3. Peek")

        print("4. Display")

        print("5. Exit")

choice = int(input("Enter your choice: "))

if choice == 1:
    value = int(input("Enter element to enqueue: "))
    queue.enqueue(value)
elif choice == 2:
    queue.dequeue()
elif choice == 3:
    queue.peek()
elif choice == 4:
    queue.display()
elif choice == 5:
    print("Program terminated.")
    break
else:
    print("Invalid choice! Please try again.")
```

Output:

```
========== RESTART: C:/Users/Divya/OneDrive/Documents/DSA/lab7d.py =====

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 67
67 inserted into circular queue.

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter element to enqueue: 897
897 inserted into circular queue.

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 1768
Invalid choice! Please try again.

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 187
Invalid choice! Please try again.
```

Edit Shell Debug Options Window Help

```
5. Exit
Enter your choice: 187
Invalid choice! Please try again.

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 2
67 deleted from circular queue.

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 3
Front element: 897

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 4
Circular Queue elements: 897

----- CIRCULAR QUEUE USING SINGLY LINKED LIST -----
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter your choice: 5
Program terminated.
```

Explanation:

data → stores the element.

next → stores the address/reference of the next node.

This inserts the new node at the **rear** while maintaining the circular connection.

Error table:

S.No	Error	Rectification
1	Setting rear.next = None breaks the circular connection.	Set rear.next = front so the last node points to the first node.
2	Not setting rear = None when the only node is deleted leaves an invalid queue.	If rear == front, set rear = None.

LAB-8

1.AIM: Python program to implement a binary tree and its operations using an array and a linked list.

Program using array:

```
# Week 8: Binary Tree using Array

tree = []

def insert(value):

    tree.append(value)

    print(value, "inserted successfully")

def display():

    if len(tree) == 0:

        print("Tree is empty")

    else:

        print("Binary Tree elements:", tree)

while True:

    print("\n1. Insert")

    print("2. Display")

    print("3. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:

        value = int(input("Enter value: "))

        insert(value)

    elif choice == 2:

        display()
```

```
    elif choice == 3:
        break
    else:
        print("Invalid choice")
```

Output:

```
============ RESTART: C:/Users/Divya/OneDrive/Documents/DSA/lab8a.py

1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 34
34 inserted successfully

1. Insert
2. Display
3. Exit
Enter your choice: 2
Binary Tree elements: [34]

1. Insert
2. Display
3. Exit
Enter your choice: 3
```

Program:

```
# Binary Tree using Linked List
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
root = Node(10)
root.left = Node(20)
root.right = Node(30)
root.left.left = Node(40)
root.left.right = Node(50)
def inorder(node):
    if node:
```

```
        inorder(node.left)
        print(node.data, end=" ")
        inorder(node.right)
def preorder(node):
    if node:
        print(node.data, end=" ")
        preorder(node.left)
        preorder(node.right)
def postorder(node):
    if node:
        postorder(node.left)
        postorder(node.right)
        print(node.data, end=" ")
print("Inorder Traversal:")
inorder(root)
print("\nPreorder Traversal:")
preorder(root)
print("\nPostorder Traversal:")
postorder(root)
```

Output:

```
========== RESTART: C:\Users\Divya\OneDrive\Documents\DSA\lab8b.py
Inorder Traversal:
40 20 50 10 30
Preorder Traversal:
10 20 40 50 30
Postorder Traversal:
40 50 20 30 10
```

Explanation:

- Binary Tree Using Array: Implements a binary tree using an array to insert, display, and search elements.
- Binary Tree Using Linked List: Implements a binary tree using nodes and performs inorder, preorder, and postorder traversals.

Error table:

Error	Retification
AttributeError: 'NoneType' object has no attribute 'left'	Check if node is not None: before accessing children.
NameError: name 'Node' is not defined	Define the Node class first.
Tree is full	Increase the array size or handle overflow.
None appears in output	Display only occupied positions if required.

LAB-9

1.AIM: Python program to implement a binary tree with its operations and traversals.

Program:

```
# Binary Tree Operations and Traversals
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
def insert(root, data):
    if root is None:
        return Node(data)
    if data < root.data:
        root.left = insert(root.left, data)
    else:
        root.right = insert(root.right, data)
    return root
def inorder(root):
    if root:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)
def preorder(root):
```

```
    if root:
print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)
def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")
def levelorder(root):
    if root is None:
        print("Tree is empty")
        return
    queue = [root]
    while queue:
        node = queue.pop(0)
        print(node.data, end=" ")
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)
root = None
while True:
    print("\n\n1. Insert")
    print("2. Inorder")
```

```
print("3. Preorder")
print("4. Postorder")
print("5. Level Order")
print("6. Exit")
ch = int(input("Enter choice: "))
if ch == 1:
    value = int(input("Enter value: "))
    root = insert(root, value)
    print("Node inserted")
elif ch == 2:
    print("Inorder:")
    inorder(root)
elif ch == 3:
    print("Preorder:")
    preorder(root)
elif ch == 4:
    print("Postorder:")
    postorder(root)
elif ch == 5:
    print("Level Order:")
    levelorder(root)
elif ch == 6:
```

```
print("Program ended")
        break
    else:
        print("Invalid choice")
```

Output:

```
>
============ RESTART: C:/Users/Divya/OneDrive/Documents/DSA/lab9.py ============

1. Insert
2. Inorder
3. Preorder
4. Postorder
5. Level Order
6. Exit
Enter choice: 1
Enter value: 54
Node inserted


1. Insert
2. Inorder
3. Preorder
4. Postorder
5. Level Order
6. Exit
Enter choice: 2
Inorder:
54

1. Insert
2. Inorder
3. Preorder
4. Postorder
5. Level Order
6. Exit
Enter choice: 1
Enter value: 456
Node inserted
```

```
Node inserted


1. Insert
2. Inorder
3. Preorder
4. Postorder
5. Level Order
6. Exit
Enter choice: 3
Preorder:
54 456

1. Insert
2. Inorder
3. Preorder
4. Postorder
5. Level Order
6. Exit
Enter choice: 4
Postorder:
456 54

1. Insert
2. Inorder
3. Preorder
4. Postorder
5. Level Order
6. Exit
Enter choice: 5
Level Order:
54 456

1. Insert
2. Inorder
3. Preorder
4. Postorder
5. Level Order
6. Exit
Enter choice: 6
Program ended
```

Explanation:

Binary tree traversal is the process of visiting every node in a binary tree exactly once. It is split into two main categories: **Depth-First Search (DFS)** and **Breadth-First Search (BFS)**

Error table:

Error	Cause	Correction
Inorder is not sorted	The tree was not built using BST rules	Compare each inserted value with the current node.
RecursionError	Incorrect recursive calls or a cycle	Ensure recursive calls move to child nodes.
Infinite loop in the menu	Exit condition is missing or incorrect	Use break when the user selects Exit.
Wrong traversal result	Traversal calls are in the wrong order	Check the order of left, root, and right.

LAB-10

1.AIM: Write a Python program to implement an AVL Tree with all its operations.

Program:

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1
class AVLTree:
    # Get height of node
    def height(self, root):
        if root is None:
            return 0
        return root.height
    # Get balance factor
    def get_balance(self, root):
        if root is None:
            return 0
        return self.height(root.left) - self.height(root.right)
    # Right rotation
    def right_rotate(self, y):
        x = y.left
        T2 = x.right
        x.right = y
    y.left = T2
```

```
    y.height = 1 + max(self.height(y.left),
                       self.height(y.right))
        x.height = 1 + max(self.height(x.left),
                       self.height(x.right))
        return x
    # Left rotation
    def left_rotate(self, x):
        y = x.right
        T2 = y.left
        y.left = x
        x.right = T2
        x.height = 1 + max(self.height(x.left),
                       self.height(x.right))
        y.height = 1 + max(self.height(y.left),
                       self.height(y.right))
        return y
    # Insert operation
    def insert(self, root, key):
        if root is None:
            return Node(key)
        if key < root.key:
            root.left = self.insert(root.left, key)
elif key > root.key:
            root.right = self.insert(root.right, key)
        else:
            return root
        root.height = 1 + max(self.height(root.left),
```

```
                        self.height(root.right))
        balance = self.get_balance(root)
        # LL Case
        if balance > 1 and key < root.left.key:
            return self.right_rotate(root)
        # RR Case
        if balance < -1 and key > root.right.key:
            return self.left_rotate(root)
        # LR Case
        if balance > 1 and key > root.left.key:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)
        # RL Case
        if balance < -1 and key < root.right.key:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)
        return root
    # Find minimum value node
    def min_value_node(self, root):
        current = root
        while current.left is not None:
            current = current.left
        return current
# Delete operation
    def delete(self, root, key):
        if root is None:
```

```
        return root
    if key < root.key:
        root.left = self.delete(root.left, key)
    elif key > root.key:
        root.right = self.delete(root.right, key)
    else:
        if root.left is None:
            return root.right
        elif root.right is None:
            return root.left
        temp = self.min_value_node(root.right)
        root.key = temp.key
        root.right = self.delete(root.right, temp.key)
    if root is None:
        return root
    root.height = 1 + max(self.height(root.left),
                          self.height(root.right))
    balance = self.get_balance(root)
    # LL Case
    if balance > 1 and self.get_balance(root.left) >= 0:
        return self.right_rotate(root)
    # LR Case
    if balance > 1 and self.get_balance(root.left) < 0:
        root.left = self.left_rotate(root.left)
        return self.right_rotate(root)
# RR Case
```

```
        if balance < -1 and self.get_balance(root.right) <= 0:
            return self.left_rotate(root)
        # RL Case
        if balance < -1 and self.get_balance(root.right) > 0:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)
        return root
    # Search operation
    def search(self, root, key):
        if root is None or root.key == key:
            return root
        if key < root.key:
            return self.search(root.left, key)
        return self.search(root.right, key)
    # Inorder traversal
    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.key, end=" ")
            self.inorder(root.right)
    # Preorder traversal
    def preorder(self, root):
        if root:
            print(root.key, end=" ")
    self.preorder(root.left)
            self.preorder(root.right)
```

```
    # Postorder traversal
    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.key, end=" ")
    # Display tree
    def display(self, root, level=0):
        if root:
            self.display(root.right, level + 1)
            print("    " * level + str(root.key))
            self.display(root.left, level + 1)
# Main program
tree = AVLTree()
root = None
while True:
    print("\n\n--- AVL TREE OPERATIONS ---")
    print("1. Insert")
    print("2. Delete")
    print("3. Search")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Display Tree")
    print("8. Exit")
```

```
choice = int(input("Enter your choice: "))
  if choice == 1:
     key = int(input("Enter value to insert: "))
     root = tree.insert(root, key)
     print("Node inserted successfully!")
  elif choice == 2:
     key = int(input("Enter value to delete: "))
     root = tree.delete(root, key)
     print("Deletion operation completed.")
  elif choice == 3:
     key = int(input("Enter value to search: "))
     if tree.search(root, key):
        print("Element found!")
     else:
        print("Element not found!")
  elif choice == 4:
     print("Inorder Traversal:")
     tree.inorder(root)
  elif choice == 5:
     print("Preorder Traversal:")
     tree.preorder(root)
  elif choice == 6:
     print("Postorder Traversal:")
     tree.postorder(root)
  elif choice == 7:
     print("AVL Tree:")
     tree.display(root)
```

```
elif choice == 8:
    print("Exiting program...")
    break
else:
    print("Invalid choice!")
```

Output:

```
========== RESTART: C:/Users/Divya/OneDrive/Documents/DSA/lab11.py ==========


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 1
Enter value to insert: 45
Node inserted successfully!


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 2
Enter value to delete: 45
Deletion operation completed.


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 3
Enter value to search: 45
Element not found!


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 54
Invalid choice!


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 1
Enter value to insert: 34
Node inserted successfully!


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
```

```
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 1
Enter value to insert: 43
Node inserted successfully!


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 1
Enter value to insert: 32
Node inserted successfully!


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 1
Enter value to insert: 76
Node inserted successfully!


--- AVL TREE OPERATIONS ---
```

```
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 5
Preorder Traversal:
34 32 76 43 98

--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 6
Postorder Traversal:
32 43 98 76 34

--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 7
AVL Tree:
        98
    76
        43
34
    32
```

```
    32


--- AVL TREE OPERATIONS ---
1. Insert
2. Delete
3. Search
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Display Tree
8. Exit
Enter your choice: 8
Exiting program...
>
```

Explanation:

An AVL Tree is a self-balancing Binary Search Tree (BST). It maintains balance after insertion and deletion using four rotations: LL, RR, LR and RL.

Operations:

1. Insertion – Adds a new node.
2. Deletion – Removes a node.
3. Searching – Finds an element.
4. Inorder – Displays elements in ascending order.
5. Preorder – Displays root, left and right.
6. Postorder – Displays left, right and root.
7. Display – Shows the tree structure.
8. Balance Factor:
9. [BF = Height(Left)-Height(Right)]

Errortable:

S.No	Error	Solution
1	SyntaxError	Correct syntax
2	IndentationError	Maintain indentation
3	ValueError	Enter valid input
4	RecursionError	Check recursive conditions

LAB-11

1.Aim: Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing).

Program:

```
#Separate Chaining

size = 10

hash_table = [[] for i in range(size)]

def insert(key):

    index = key % size

    hash_table[index].append(key)

def display():

    for i in range(size):

        print(i, ":", hash_table[i])

#Main program

n = int(input("Enter number of elements: "))

    for i in range(n):

        key = int(input("Enter key: "))

        insert(key)

print("\nHash Table:")

display()
```

Output:

```
Enter "help" below or click "Help" above for more information.
>>
========== RESTART: C:/Users/Divya/OneDrive/Documents/DSA/lab10a.py ==========
Enter number of elements: 6
Enter key: 45
Enter key: 54
Enter key: 43
Enter key: 87
Enter key: 332
Enter key: 988

Hash Table:
0 : []
1 : []
2 : [332]
3 : [43]
4 : [54]
5 : [45]
6 : []
7 : [87]
8 : [988]
9 : []
```

Program:

```
#Linear Probing

size = 10

hash_table = [-1] * size

def insert(key):

  index = key % size


   while hash_table[index] != -1:
      index = (index + 1) % size


   hash_table[index] = key


def display():

   for i in range(size):

     print(i, ":", hash_table[i])

#Main program

n = int(input("Enter number of elements: "))

    for i in range(n):

       key = int(input("Enter key: "))

        insert(key)

print("\nHash Table:")

display()
```

Output:

```
========== RESTART: C:/Users/Divya/OneDrive/Documents/DSA/lab10b.py ======
Enter number of elements: 5
Enter key: 43
Enter key: 09
Enter key: 32
Enter key: 76
Enter key: 67

Hash Table:
0 : -1
1 : -1
2 : 32
3 : 43
4 : -1
5 : -1
6 : 76
7 : 67
8 : -1
9 : 49
```

Program:

```
#Quadratic Probing

size = 10

hash_table = [-1] * size

def insert(key):

    index = key % size

    i = 0


    while i < size:
        new_index = (index + i * i) % size
        if hash_table[new_index] == -1:
            hash_table[new_index] = key
            return
        i += 1
    print("Hash Table is Full. Cannot insert", key)


def display():

    for i in range(size):

        print(i, ":", hash_table[i])

#Main program

n = int(input("Enter number of elements: "))

    for i in range(n):

        key = int(input("Enter key: "))

    insert(key)

print("\nHash Table:")

display()
```

Output:

```
========== RESTART: C:/Users/Divya/OneDrive/Documents/DSA/lab10c.py ========
Enter number of elements: 4
Enter key: 56
Enter key: 90
Enter key: 23
Enter key: 87

Hash Table:
0 : 90
1 : -1
2 : -1
3 : 23
4 : -1
5 : -1
6 : 56
7 : 87
8 : -1
9 : -1
```

Explanation:

Hashing is a technique used in Data Structures to store and retrieve data quickly using a special function called a Hash Function.

A hash function converts a given key into an index of a hash table.

Error table:

S.No.	Error	Rectification
1	IndexError	Check the index and table size
3	ValueError	Enter valid integer values
4	Infinite Loop	Check for an empty position
5	KeyError	Check whether the key exists
6	Collision	Use collision resolution techniques